

Ok I see a tiny part of what’s happening. It’s only a very small piece of the puzzle. It’s maybe the simplest non-trivial case, and it only says something about the underlying topology/geometry. It doesn’t clarify anything about the coordinate rings or structure sheaves living on these spaces, which is to say that iit doesn’t say much about the log semirings living over / guiding the topology in the general case, but maybe it start to hint at where/what they might be.

0.1 Main observation

So here’s what I noticed. I’m just stating it succinctly. You’ll have to think through details:

Observation 0.1.1. Consider the processes of setting up and running binary search on an ordered set $\mathcal{S}$ containing 2^N elements, i.e., on a `list` of length 2^N .

We know that there are two related but very different structures we can build on $\mathcal{S}$ to run binary search:

1. *[Mal24]-style quotient tower.*
2. *Binary search tree*

The apparent duality between these two objects is actually much deeper than just two possible choices, and is more like *Gelfand duality*, that is, duality between spaces and rings via Spec:

1. *The [Mal24]-style quotient tower* is a finite piece of the inverse system of discrete sets that induces the profinite, inverse limit topology on the ring of 2-adic (or *dyadic*) integers $\widehat{\mathbb{Z}}_2$. For this particular problem of binary search, our quotient tower identifies our set o nodes $\mathcal{S}$ itself with the quotient $\mathbb{Z}/2^N\mathbb{Z}$:

$$\widehat{\mathbb{Z}}_2 \twoheadrightarrow \mathbb{Z}/2^N\mathbb{Z} \cong \mathcal{S}$$

2. *The binary search tree* is a piece of the Berkovich analytic space associated to $\widehat{\mathbb{Z}}_2$, the space denoted “ $\mathcal{M}(\widehat{\mathbb{Z}}_2)$ ” in the Berkovich literature. The Berkovich space *is* the space being topologized by the inverse system quotient tower.

So the picture that is emerging is one where, for binary search at least, we’re equipping a list we need to search through with the topology of a Berkovich space so that we can move through in logarithmic time.

So the em vague general picture I have is one where we’re doing something *like* a synthetic “Berkovich-like topologization” of our graph so that the graph’s nodes embed into a dual, hidden “Berkovich-like” space (not sure how much such a thing actually exists in our case) that we traverse really quickly.¹

— Tyler

0.2 A bit of a mess below, but lots of important pieces

The guiding algebro-geometric example here is $\mathbb{G}_m$

Below is some output from GPT 5.5 that clarifies the role of the logarithm in all this. The drawings below are mine, except the Lionel Lang image, which comes from a paper that’s not mine.

The shared concrete mechanism is:

¹ Just an aside: *duality* is one of the most important things going on in tropical geometry: <https://pubs.ams.org/journals/jams/2018-31-02/S0894-0347-2017-00890-7>

multiplicative scale $\longrightarrow$ additive depth,
fine metric detail $\longrightarrow$ coarse order or valuation data,
ordinary geometry $\longrightarrow$ tree, fan, quotient skeleton.

For tropicalization with $0 < b < 1$, the meaningful expression is not a fixed $\log_b(x)$ as $b \rightarrow 0$, since fixed x mostly degenerates. The meaningful move is to write

$$x(b) = b^a,$$

so that

$$\log_b x(b) = a.$$

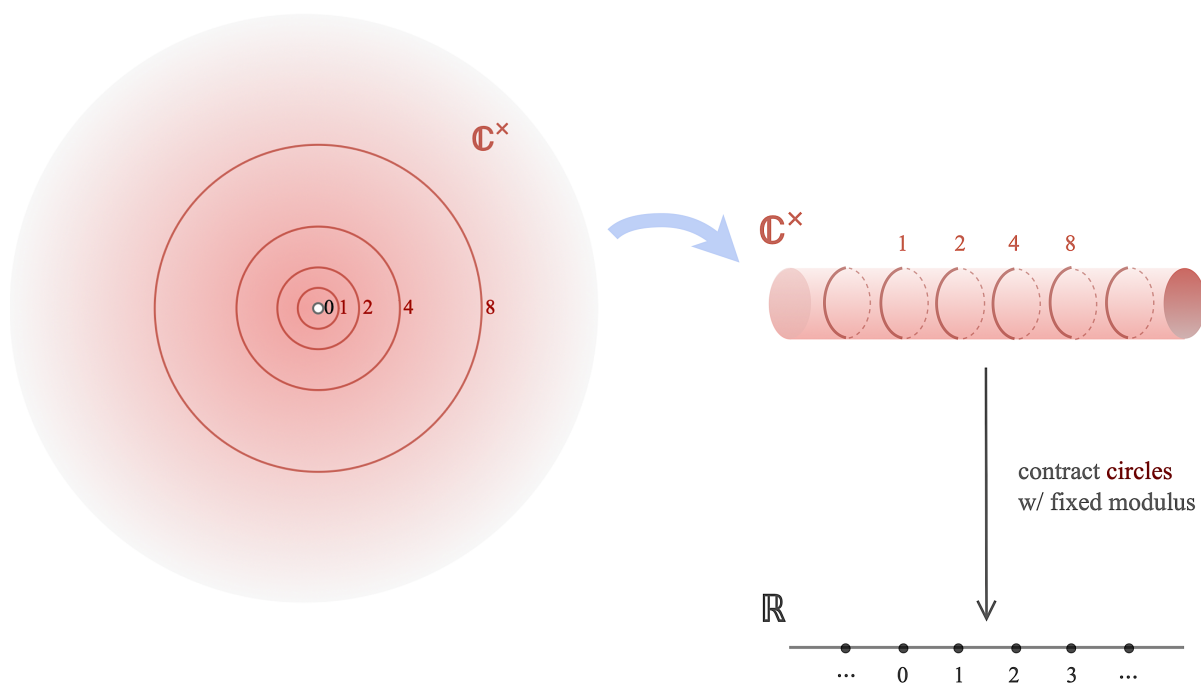


Figure 1: [...]

For sums, one obtains

$$\log_b(b^a + b^c) \longrightarrow \min(a, c) \quad \text{as } b \rightarrow 0.$$

Lionel Lang

Figure 2: From a paper by Lionel Lang: Applying $\log || - ||$ to curves in a toric variety tend to produce tropical-looking things that aren't quite polyhedral, and look sort of like biological amoebas posing as polyhedral geometry.

Thus tropicalization says:

keep the exponent or scale;
throw away coefficient-level detail;
turn addition into $\min$;

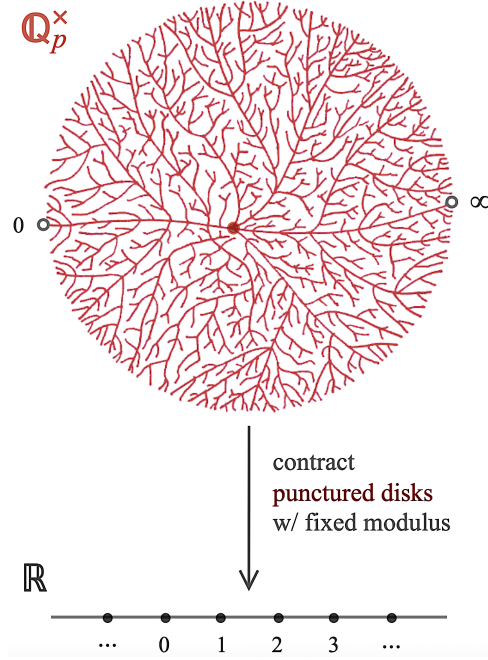


Figure 3: [...]

turn multiplication into addition.

Now compare the binary contraction tower. Take a dyadic grid. At depth k , cells have size

$$2^{-k}.$$

A binary quotient or coset tier forgets all detail below that scale. A point is no longer “this exact grid point”; it is “this dyadic coset at depth k .” The important invariant becomes

$$\text{scale} = 2^{-k},$$

and therefore

$$\text{depth} = -\log_2(\text{scale}) = k.$$

So the binary tower is also doing:

keep the exponent or depth;
throw away within-cell detail.

That is the same move as tropicalization.

The deep reason these feel like the same phenomenon is that both are valuation machines. A valuation converts size or order into an additive number:

$$v(xy) = v(x) + v(y),$$

and

$$v(x + y) \geq \min(v(x), v(y)).$$

In the tropical limit, this inequality becomes the governing rule:

$$v(x + y) = \min(v(x), v(y))$$

generically.

In the binary coset tower, the valuation is dyadic depth:

$$v_2(\text{scale}) = \text{depth}.$$

Equivalently, for two grid points,

$$\text{distance scale} \sim 2^{-k} \quad \Longleftrightarrow \quad \text{separation depth} \sim k.$$

Two points are indistinguishable until the tier where their binary addresses split. The tower keeps that split-depth and discards the lower bits. That is a tropical or ultrametric kind of geometry.

So a useful explanation is:

Tropicalization and binary contraction towers are both ways of replacing metric magnitude by scale depth. The logarithm is not an extra analytic trick; it is the coordinate change from multiplicative refinement to additive tier index.

Or, more compactly:

$$\text{tropicalization:} \quad x = b^a \longmapsto a,$$

$$\text{binary tower:} \quad \text{cell size} = 2^{-k} \longmapsto k.$$

Both say: the real object is not the tiny quantity itself, but the exponent at which it lives.

That is why $\lim_{b \rightarrow 0} \log_b$ and a tower of binary cosets can be understood as manifestations of the same idea: each builds a limiting skeleton where only dominance, scale, and ancestry survive.

References

- [Mal24] Abdullah Naeem Malik. *Simplicial Methods in Graph Machine Learning*. Ph.d. dissertation, Florida State University, 2024.